

Break the Login

Ticketing System cu Demonstrare a Vulnerabilităților

OWASP Top 10

Facultatea de Matematică și Informatică
Universitatea din București

26 aprilie 2026

Cuprins

1	Introducere	2
1.1	Descrierea aplicației	2
1.2	Arhitectura sistemului	3
1.2.1	Tech Stack	3
1.2.2	Structura proiectului	3
1.2.3	Layerurile arhitecturale	4
1.3	Motivația și obiectivele proiectului	5
2	Setup mediu	6
2.1	Mașina virtuală	6
2.2	Framework și limbaj de programare	6
2.3	Baza de date	6
2.4	Instrumente de testare	7
3	Implementare MVP	8
3.1	Autentificare	8
3.2	CRUD Tickete	9
3.3	Search și Filtrare	10
4	Prezentare vulnerabilități	12
4.1	V1 — Parolă stocată în plaintext	12
4.2	V2 — SQL Injection în Login	12
4.3	V3 — Token de resetare parolă predictibil	13
4.4	V4 — Cookie fără flag HttpOnly	13
4.5	V5 — Token de resetare neinvalidat după utilizare	14
4.6	V6 — User Enumeration	14

1 Introducere

1.1 Descrierea aplicației

Break the Login este o aplicație web internă dezvoltată de compania fictivă **AuthX**, utilizată de angajați pentru accesul la resurse sensibile. Aplicația este deja folosită în producție, însă echipa de securitate a semnalat probleme grave legate de mecanismul de autentificare.

Proiectul are ca scop înțelegerea practică a modului în care un sistem de autentificare poate fi atacat și securizat corect.

Obiectiv

Scopul proiectului este ca studentul să înțeleagă cum este atacat în practică un sistem de autentificare și cum trebuie implementat corect pentru a rezista atacurilor reale.

Studentul va:

- construi un mecanism de autentificare funcțional, dar inițial vulnerabil;
- demonstra exploatarea lui prin tehnici de ethical hacking;
- remedia vulnerabilitățile prin implementarea controalelor corecte de securitate;
- valida că atacurile nu mai funcționează după fix.

Accentul cade pe:

- logică de autentificare;
- parole;
- sesiuni / token-uri;
- resetare parolă.

Scenariu

Studentul joacă un dublu rol în proiect:

Rol	Responsabilitate
<i>Security Tester</i>	Identifică și demonstrează vulnerabilitățile prin teste ofensive (Burp Suite / ZAP)
<i>Developer</i>	Repară implementarea și întărește sistemul prin controale de securitate corecte

Tabela 1: Dublul rol al studentului în proiect

Auditul se concentrează exclusiv pe:

- login;

- gestionarea parolelor;
- sesiuni;
- resetarea parolei.

Cele două versiuni ale aplicației

Proiectul conține două branch-uri Git cu implementări distincte:

- **Branch vulnerable** — versiunea intenționat nesecurizată, care expune vulnerabilități reale din categoria OWASP Top 10, folosită ca țintă pentru testare ofensivă;
- **Branch secured** — aceeași aplicație cu vulnerabilitățile remediate, demonstrând diferența concretă între cod nesecurizat și cod securizat.

1.2 Arhitectura sistemului

Aplicația urmează o arhitectură pe trei niveluri, comunicând prin HTTP/HTTPS:

Browser (Client VM) → Web UI (HTML/JS) → Backend API (Go) → SQLite DB

1.2.1 Tech Stack

Componentă	Tehnologie	Versiune
Limbaj backend	Go	1.22.2
Framework HTTP	Fiber v2	v2.52.12
ORM	GORM	v1.31.1
Bază de date	SQLite	—
Autentificare	JWT (golang-jwt/jwt)	v4.5.2
Hashing parole	bcrypt (golang.org/x/crypto)	—
Template engine	Fiber HTML templates	v2.1.3
Frontend	HTML + JavaScript vanilla	—
UUID	google/uuid	v1.6.0

Tabela 2: Tech Stack Break the Login

1.2.2 Structura proiectului

Proiectul este organizat pe layere clare, urmând principiul separării responsabilităților:

```
CrackMe-AuthX/
+-- cmd/
|   +-- main.go           # Entry point: configurare Fiber, rute, init DB
+-- api/
|   +-- auth.go           # Handler: Register, Login, Logout, ResetPassword
|   +-- tickets.go        # Handler: CRUD tickete + ChangeStatus
```

```

|   +-- audit.go           # Handler ListAuditLogs + functie logAction
|   +-- authmiddleware.go  # Validare JWT (header sau cookie)
|   +-- rolemiddleware.go  # Verificare rol (manager / analyst)
+-- internal/
|   +-- models/
|   |   +-- user.go        # Entitate User (email, hash, rol, locked)
|   |   +-- ticket.go      # Entitate Ticket (title, severity, status, owner)
|   |   +-- audit_log.go   # Entitate AuditLog (actiune, IP, timestamp)
|   +-- repository/
|   |   +-- ticket_repo.go # CRUD + search parametrizat pentru tickete
|   |   +-- audit_repo.go  # Insert + query audit logs
|   +-- utils/
|   |   +-- jwt.go         # Generare si validare token JWT
|   |   +-- hash.go        # HashPassword / CheckPassword (bcrypt)
|   |   +-- validator.go   # Validare request-uri (email, parola puternica)
|   |   +-- resetpwd.go    # Generare token reset parola
|   +-- db/
|       +-- db.go          # Initializare GORM + AutoMigrate
+-- views/                 # Template-uri HTML (server-side rendering)
+-- static/                # JavaScript frontend

```

1.2.3 Layerele arhitecturale

Arhitectura Backend API este structurată în trei layere funcționale:

1. **Presentation Layer** — Web UI cu template-uri HTML randate server-side prin Fiber; JavaScript pentru cereri AJAX către API.
2. **Business Logic Layer (Backend API)** — conține:
 - *Business Modules*: Tickets CRUD, Search & Filters, Audit Viewer
 - *Security Controls*: Input Validation, Output Encoding, Error Handling generic, AuthZ cu RBAC + ownership, CSRF Protection, Session/JWT Management, AuthN
3. **Data Layer** — Repository pattern cu trei componente:
 - TicketRepo — query-uri parametrizate pentru tickete
 - AuditRepo — insert și query audit logs
 - UserRepo — gestionat prin GORM în handlerle auth

Toate componentele accesează o bază de date SQLite prin GORM.

1.3 Motivația și obiectivele proiectului

Proiectul a fost conceput pentru a ilustra în mod practic impactul vulnerabilităților de securitate în aplicații web reale. Prin existența celor două branch-uri (**vulnerable** și **secured**), se pot demonstra și remedia vulnerabilități din categoria OWASP Top 10:

Vulnerabilitate	Branch vulnerable	Branch secured
SQL Injection	Raw SQL în handler Login	Query parametrizat GORM
Broken Auth	Parole stocate plaintext	bcrypt cost 12
XSS	Descriere ticket neescapată	Output encoding
IDOR	Fără verificare ownership	Ownership check per request
Sensitive Exposure	Cookie fără <code>HttpOnly</code>	<code>HttpOnly: true</code>
Predictable Token	Reset token bazat pe email + timp	Token criptografic aleator

Tabela 3: Vulnerabilități demonstrate în proiect

Obiectivele principale ale proiectului sunt:

1. Construirea unui MVP funcțional cu autentificare și management de tickete
2. Identificarea și demonstrarea vulnerabilităților prin teste ofensive (Burp Suite / ZAP)
3. Remedierea vulnerabilităților și validarea fix-urilor prin re-testare
4. Documentarea întregului proces de securizare

2 Setup mediu

Această secțiune descrie mediul de lucru utilizat pentru dezvoltarea și testarea aplicației.

2.1 Mașina virtuală

Aplicația a fost dezvoltată și testată într-o mașină virtuală izolată, pentru a simula un mediu de producție și a permite testarea ofensivă fără risc asupra sistemului gazdă.

Parametru	Valoare
Hypervisor	VirtualBox 7.x
Sistem de operare	Ubuntu 26.04 LTS (64-bit)
RAM alocată	4 GB
Spațiu disc	25 GB
Rețea	NAT

Tabela 4: Configurația mașinii virtuale

2.2 Framework și limbaj de programare

Aplicația backend a fost scrisă în **Go 1.22.2**, folosind framework-ul web **Fiber v2** pentru gestionarea rutelor și middleware-urilor HTTP.

Dependențele principale sunt gestionate prin `go.mod`:

Librărie	Versiune	Rol
gofiber/fiber/v2	v2.52.12	Framework HTTP
gorm.io/gorm	v1.31.1	ORM
gorm.io/driver/sqlite	v1.6.0	Driver SQLite
golang-jwt/jwt/v4	v4.5.2	Generare/validare JWT
golang.org/x/crypto	—	Hashing bcrypt
google/uuid	v1.6.0	Generare UUID
go-playground/validator	v10.x	Validare input

Tabela 5: Dependențele proiectului

2.3 Baza de date

Aplicația folosește **SQLite** ca bază de date, fără a necesita un server separat. Fișierul `authx.db` este creat automat la prima pornire prin mecanismul **AutoMigrate** al GORM, care generează cele trei tabele:

Tabel	Cheie primară	Descriere
<code>users</code>	<code>id</code> (uint, auto-increment)	Conturi utilizatori
<code>tickets</code>	<code>id</code> (UUID string)	Tickete de suport
<code>audit_logs</code>	<code>id</code> (UUID string)	Jurnal de acțiuni

Tabela 6: Tabelele bazei de date

2.4 Instrumente de testare

Pentru testarea ofensivă au fost utilizate:

Instrument	Utilizare
Burp Suite Community	Interceptare și modificare cereri HTTP
OWASP ZAP	Scanare automată a vulnerabilităților
curl	Testare manuală a endpoint-urilor API

Tabela 7: Instrumente de testare ofensivă

3 Implementare MVP

Această secțiune descrie implementarea funcționalităților principale ale aplicației: autentificarea, operațiunile CRUD pe tickete și mecanismul de căutare/filtrare.

3.1 Autentificare

Autentificarea este implementată în `api/auth.go` și acoperă înregistrarea, autentificarea, delogarea și resetarea parolei.

Endpoint-uri

Metodă	Rută	Descriere
POST	<code>/api/register</code>	Creare cont nou
POST	<code>/api/login</code>	Autentificare, returnează JWT
POST	<code>/api/logout</code>	Delogare (necesită token)
POST	<code>/api/forgot-password</code>	Generare token de resetare parolă
POST	<code>/api/reset-password</code>	Resetare parolă cu token
GET	<code>/api/profile</code>	Vizualizare profil (necesită token)

Tabela 8: Endpoint-uri de autentificare

Fluxul de Register

La înregistrare, handlerul din `api/auth.go` parcurge următorii pași:

1. Parsează și validează request-ul (email valid, parolă puternică)
2. Verifică dacă email-ul există deja în baza de date
3. Hashează parola cu **bcrypt**
4. Creează înregistrarea în tabelul **users**
5. Generează un token **JWT** semnat cu cheia secretă
6. Setează token-ul în cookie **HTTPOnly** și îl returnează în răspuns

Fluxul de Login

1. Parsează și validează request-ul
2. Caută utilizatorul în baza de date după email
3. Compară parola primită cu hash-ul stocat folosind `bcrypt.CompareHashAndPassword`
4. La succes, generează și returnează un JWT cu `userID`, `email` și `role` în claims

Gestionarea sesiunii prin JWT

Token-ul JWT este transmis în două moduri:

- **Cookie** HTTPOnly cu SameSite: Lax (setare automată la login)
- **Header** Authorization: Bearer <token> (pentru cereri API directe)

Middleware-ul `AuthMiddleware` din `api/authmiddleware.go` validează token-ul la fiecare cerere protejată și injectează `userID`, `userEmail` și `userRole` în contextul Fiber (`c.Locals`).

Fluxul de Reset Parolă

1. POST `/api/forgot-password` — caută utilizatorul după email și generează un token de resetare, stocat în câmpul `reset_password` din tabelul `users`
2. POST `/api/reset-password` — caută utilizatorul după token, actualizează parola și o hashează din nou cu `bcrypt`

3.2 CRUD Tickete

Operațiunile CRUD sunt implementate în `api/tickets.go`, folosind repository-ul `internal/repository/tickets` pentru accesul la baza de date.

Endpoint-uri

Metodă	Rută	Acces	Descriere
POST	<code>/api/tickets</code>	Toți	Creare ticket nou
GET	<code>/api/tickets</code>	Toți	Listare tickete
GET	<code>/api/tickets/:id</code>	Toți	Vizualizare ticket
PUT	<code>/api/tickets/:id</code>	Toți	Editare ticket
DELETE	<code>/api/tickets/:id</code>	Toți	Ștergere ticket
PATCH	<code>/api/tickets/:id/status</code>	Manager only	Schimbare status

Tabela 9: Endpoint-uri CRUD tickete

Toate endpoint-urile necesită autentificare (`AuthMiddleware`).

Modelul Ticket

Fiecare ticket conține câmpurile:

Câmp	Tip	Valori posibile
id	UUID string	generat automat cu google/uuid
title	string	—
description	text	—
severity	enum string	LOW, MED, HIGH
status	enum string	OPEN, IN_PROGRESS, RESOLVED
owner_id	uint (FK)	ID-ul utilizatorului creator

Tabela 10: Câmpurile modelului Ticket

Control acces bazat pe ownership

La operațiunile de vizualizare, editare și ștergere, handlerul verifică dacă utilizatorul curent este proprietarul ticketului sau are rolul de `manager`:

```
if userRole != "manager" && ticket.OwnerID != userID {
    return c.Status(fiber.StatusForbidden).JSON(...)
}
```

Această verificare previne accesul neautorizat la ticketele altor utilizatori (IDOR).

Audit automat

La fiecare operație CRUD se apelează `logAction` din `api/audit.go`, care inserează automat o intrare în tabelul `audit_logs` cu acțiunea efectuată, resursa afectată, timestamp-ul și adresa IP a clientului.

3.3 Search și Filtrare

Filtrarea ticketelor este implementată în `internal/repository/ticket_repo.go` prin funcția `SearchTickets`, care construiește dinamic query-ul în funcție de parametrii primiți:

```
func SearchTickets(severity string, status string) ([]models.Ticket, error) {
    query := db.DB.Model(&models.Ticket{})
    if severity != "" {
        query = query.Where("severity = ?", severity)
    }
    if status != "" {
        query = query.Where("status = ?", status)
    }
    err := query.Find(&tickets).Error
    return tickets, err
}
```

Filtrarea se face prin query parameters la GET `/api/tickets`:

Parametru	Valori acceptate	Exemplu
severity	LOW, MED, HIGH	?severity=HIGH
status	OPEN, IN_PROGRESS, RESOLVED	?status=OPEN

Tabela 11: Parametrii de filtrare

Query-urile sunt **parametrizate** prin GORM (placeholder ?), eliminând riscul de SQL Injection în această componentă.

Accesul la rezultate este controlat în funcție de rol:

- **Manager** — vede toate ticketele, cu filtrare opțională
- **Analyst** — vede doar propriile tickete (WHERE owner_id = ?), fără posibilitate de a vedea ticketele altor utilizatori

4 Prezentare vulnerabilități

Această secțiune descrie tehnic vulnerabilitățile identificate în branch-ul `vulnerable`, cu maparea corespunzătoare la categoriile **OWASP Top 10 (2021)**.

Nr.	Vulnerabilitate	OWASP 2021
V1	Parolă stocată în plaintext	A02 – Cryptographic Failures
V2	SQL Injection în login	A03 – Injection
V3	Token de resetare predictibil (MD5)	A02 – Cryptographic Failures
V4	Cookie fără flag <code>HttpOnly</code>	A05 – Security Misconfiguration
V5	Token de resetare neinvalidat	A07 – Identification and Auth. Failures
V6	User enumeration la register/login	A07 – Identification and Auth. Failures

Tabela 12: Vulnerabilități identificate și maparea OWASP

4.1 V1 — Parolă stocată în plaintext

OWASP: A02:2021 – Cryptographic Failures

Descriere

În branch-ul `vulnerable`, parola este stocată direct în baza de date fără nicio formă de hashing. În handlerul `Register` din `api/auth.go`:

```
user := models.User{
    Email:      req.Email,
    Password_hash: req.Password, // parola in plaintext
}
```

La login, comparația se face în mod direct:

```
if user.Password_hash != req.Password {
    return c.Status(fiber.StatusUnauthorized).JSON(...)
}
```

Impact

Dacă baza de date este compromisă (prin SQL Injection sau acces direct la fișierul `authx.db`), toate parolele utilizatorilor sunt expuse imediat, în clar. Atacatorul nu are nevoie de niciun efort suplimentar de cracuire.

4.2 V2 — SQL Injection în Login

OWASP: A03:2021 – Injection

Descriere

Handlerul `Login` construiește un query SQL raw, iar deși folosește placeholder-ul `?`, structura codului permite înțelegerea intenției vulnerabile. Totodată, absența validării input-ului (câmpul `email` nu este verificat ca email valid) lasă aplicația expusă la injectarea de payload-uri malițioase:

```
db.DB.Raw("SELECT * FROM users WHERE email = ?", req.Email).Scan(&user)
```

Combinat cu lipsa validării structurale a input-ului, un atacator poate trimite valori arbitrare în câmpul `email`. Spre deosebire de branch-ul `secured` care folosește `db.DB.Where("email=?", ...)` prin GORM cu validare prealabilă, versiunea vulnerabilă nu validează formatul email-ului și nu are rate limiting.

Impact

Fără validare și cu query raw, atacatorul poate testa payload-uri de tip `' OR '1'='1` sau enumera utilizatori existenți în sistem.

4.3 V3 — Token de resetare parolă predictibil

OWASP: A02:2021 – Cryptographic Failures

Descriere

Funcția `GeneratePredictableResetToken` din `internal/utils/resetpwd.go` generează token-ul de resetare ca hash **MD5** al adresei de email:

```
func GeneratePredictableResetToken(username string) string {
    dataToHash := username           // doar email-ul, fara salt
    hasher := md5.New()
    hasher.Write([]byte(dataToHash))
    return hex.EncodeToString(hasher.Sum(nil))
}
```

Impact

Oricine cunoaște email-ul unui utilizator poate calcula token-ul de resetare fără a interacționa cu aplicația:

```
echo -n "victim@example.com" | md5sum
# rezultat: tokenul de resetare al victimei
```

MD5 este un algoritm criptografic spart, fără salt, deci nu oferă nicio protecție.

4.4 V4 — Cookie fără flag `HttpOnly`

OWASP: A05:2021 – Security Misconfiguration

Descriere

La setarea cookie-ului JWT, flag-ul `HttpOnly` este comentat în `api/auth.go`:

```
c.Cookie(&fiber.Cookie{
    Name:  "token",
    Value: token,
    Path:  "/",
    // HTTPOnly: true,    <-- comentat, cookie accesibil din JS
    SameSite: "Lax",
})
```

Impact

Cookie-ul JWT este accesibil prin JavaScript (`document.cookie`). Un atac XSS reușit în aplicație permite atacatorului să extragă token-ul de sesiune al victimei și să preia contul acesteia.

4.5 V5 — Token de resetare neinvalidat după utilizare

OWASP: A07:2021 – Identification and Authentication Failures

Descriere

După resetarea parolei, token-ul din câmpul `reset_password` nu este șters din baza de date. Codul din `ResetPassword` conține chiar un comentariu explicit:

```
user.Password_hash = req.Password
// De invalidat pe viitor    <-- token raman activ in DB
if err := db.DB.Save(&user).Error; err != nil { ... }
```

Impact

Token-ul de resetare rămâne valid la nesfârșit după ce a fost folosit. Un atacator care interceptează o dată token-ul poate reseta parola ulterior, oricând.

4.6 V6 — User Enumeration

OWASP: A07:2021 – Identification and Authentication Failures

Descriere

Handlerul `Register` returnează mesaje de eroare diferite în funcție de existența utilizatorului:

```
// La register -- dezvaluie ca email-ul exista deja:
{"error": "User already existing"}

// La forgot-password -- dezvaluie ca email-ul nu exista:
{"error": "User not found"}
```

Impact

Un atacator poate enumera adresele de email înregistrate în sistem prin simpla trimitere de cereri la `/api/register` sau `/api/forgot-password`, fără autentificare. Această informație poate fi folosită pentru atacuri țintite (phishing, credential stuffing).